AUTONOMOUS MULTI-AGENT SOFTWARE FACTORY // v0.1.0

ORCHESTRA: Полное руководство пользователя

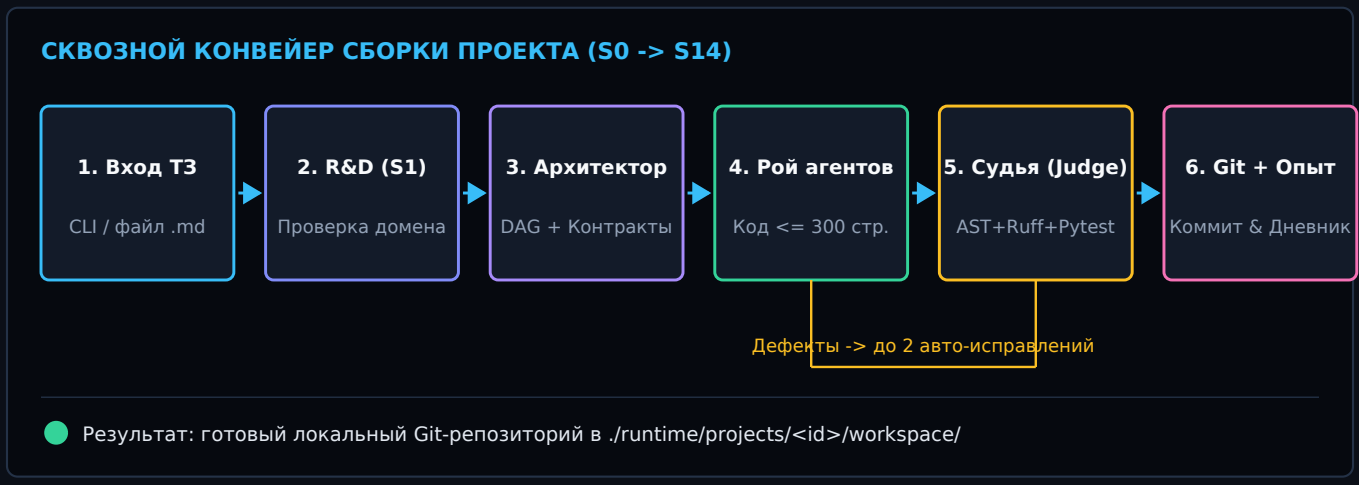
Исчерпывающая практическая инструкция на русском языке: установка, интерактивная настройка ключей и модели, постановка ТЗ, управление пулом API, Web-дашборд, Telegram-бридж, крипто-домен и все скрытые технические нюансы эксплуатации.

Ключевые возможности ORCHESTRA	Архитектурные гарантии и удобство
100% локальное хранение Весь написанный код, тесты, коммиты, граф задач и дневники агентов хранятся только у вас на компьютере в папке ./runtime/.	**Детерминированный Судья** Ни один узел не считается готовым «на слово» модели: код проверяется AST, Ruff, Pytest и доменными правилами до exit_code == 0.
Умный пул API-ключей Работает даже с 1 ключом, а при нескольких — балансирует нагрузку и мгновенно переключается на резервный ключ при ошибке HTTP 429.	**Мастер setup_env.py** Поочерёдный ввод модели и нужного числа ключей в терминале: лишние поля и слоты пропускаются обычным нажатием пробела.

№	Содержание руководства (доступно также в боковых закладках PDF)
01	Как устроен ORCHESTRA и где хранятся файлы проектов
02	Установка на Windows 11, Linux, macOS и в Docker
03	Интерактивная настройка через setup_env.py (правило пробела)
04	Пул API-ключей, лимиты HTTP 429 и выбор LLM-модели
05	Самодиагностика (doctor) и офлайн-проверка (demo)
06	Как правильно писать ТЗ и запускать сборку (orchestra run)
07	Полный справочник всех команд CLI
08	Веб-консоль оператора (serve) и ответы на эскалации
09	Telegram-бридж оператора: настройка и нюансы безопасности
10	Накопление опыта: Скиллы (/skills/) и Таланты (/agent_talents/)
11	Крипто-домен: мультибиржевая база, Decimal и защита LIVE-режима
12	Бюджеты токенов, тайм-ауты и автовосстановление после сбоев
13	Справочник переменных .env и решение типовых проблем (FAQ)

01. Как устроен ORCHESTRA и где хранятся файлы проектов

ORCHESTRA — это локальная автономная фабрика разработки программного обеспечения на собственном асинхронном движке Python (asyncio) без внешних тяжёлых фреймворков вроде LangChain или CrewAI. Вместо хаотичного «общего чата ботов» система работает как строгое конструкторское бюро: Мета-Архитектор строит ациклический граф задач (`MentalMap`), замораживает публичные интерфейсы (`ContractSignature`), раздаёт изолированные файлы профильным агентам, а приёмку ведёт независимый детерминированный Судья (`IndependentJudge`).



1.1. Где именно лежат файлы написанных проектов (приватность 100%)

Все результаты работы оркестратора сохраняются **исключительно локально на вашем компьютере** внутри рабочей директории `./runtime/` (либо в другой папке на вашем диске, которую вы укажете в переменной `ORCHESTRA_RUNTIME_DIR`). Ни один файл сгенерированного проекта никогда автоматически не загружается в облако или чужой GitHub-репозиторий.

Структура локального хранения на вашем диске

```
razrabotka_orkestra/                # Папка самой программы ORCHESTRA
├── .env                            # Ваши локальные ключи (в .gitignore, права 0600)
├── setup_env.py                    # Интерактивный мастер ввода модели и ключей
└── runtime/                        # ВСЕ ДАННЫЕ И НАПИСАННЫЕ ПРОЕКТЫ (в .gitignore!)
    ├── projects/
    │   └── <project_id>/           # Папка конкретного заказанного вами проекта
    │       ├── workspace/          # <-- ГОТОВЫЙ КОД ПРОЕКТА + свой локальный .git!
    │       ├── mental_map.json     # Граф узлов, статусы и чексумма SHA-256
    │       ├── mental_map.json.1.bak# Ротируемые резервные копии состояния (до 5 шт.)
    │       ├── events.jsonl        # Неизменяемый журнал всех шагов (хеш-цепочка)
    │       └── rpc_messages.jsonl  # Аудит межагентных запросов
    ├── agent_talents/<role_id>/    # Накопленный опыт профессий и DISCOVERY_DIARY.md
    ├── skills/                     # Исполняемые скиллы (manifest.json + тесты)
    ├── cache/                      # Кэш capability-проб моделей и состояния пула
    └── docs_cache/                 # Локальный офлайн-кэш документации для R&D
```

> ● **ВАЖНЫЙ НЮАНС: Два разных Git-репозитория — не перепутайте их!**
> У самого оркестратора есть свой корневой `.git`, а у каждого написанного им проекта внутри `runtime/projects/<id>/workspace/.git` автоматически создаётся **собственный отдельный Git-репозиторий** на ветке `orchestra/<id>`. Благодаря этому генерируемый агентами код никогда не засоряет и не ломает код самого оркестратора. Когда проект готов, вы можете просто скопировать папку `workspace/` куда угодно или зайти в неё и выполнить `git remote add origin <ваш_URL> && git push`.

1.2. Что вообще передаётся по сети?

- **В офлайн-режиме (--mock / demo)**: **0 байт в интернет**. Никакие внешние серверы не вызываются, даже подсчёт токенов работает офлайн.
- **В боевом режиме (openai_compat)**: по HTTPS отправляются только промпты к выбранному вами LLM-серверу (например, `integrate.api.nvidia.com`). Перед логированием все секреты вырезаются потоковым фильтром `SecretRedactor`.
- **Telegram (по умолчанию ВЫКЛЮЧЕН)**: соединяется с `api.telegram.org` только если вы сами ввели токен бота и свой числовой User ID.

02. Установка на Windows 11, Linux, macOS и в Docker

Для работы требуется Python версии 3.11 или 3.12 и установленный Git (>= 2.35). Все зависимости строго зафиксированы точными версиями в файлах `requirements.txt` и `requirements-dev.txt`.

2.1. Установка на Linux и macOS

Терминал Linux / macOS

```
git clone https://github.com/AlexP0-sol/razrabotka_orkestra.git
cd razrabotka_orkestra
git checkout arena/77f5f735-razrabotka-orkestra

# Создаёт виртуальное окружение .venv и ставит все пакеты:
make install
```

2.2. Установка на Windows 11 (PowerShell) — важные нюансы

На стандартной Windows утилита make часто не установлена по умолчанию, поэтому используйте прямые команды PowerShell ниже. Оркестратор изначально спроектирован под полноценную нативную работу на Windows 11:

Windows 11 PowerShell

```
git clone https://github.com/AlexP0-sol/razrabotka_orkestra.git
cd razrabotka_orkestra
git checkout arena/77f5f735-razrabotka-orkestra

# 1. Создаём виртуальное окружение Python 3.11 / 3.12:
py -3.11 -m venv .venv

# 2. Разрешаем активацию скриптов в текущем окне PowerShell и активируем .venv:
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1

# 3. Устанавливаем зависимости и сам пакет orchestra:
python -m pip install --upgrade pip
pip install -r requirements-dev.txt -e .
```

● ВАЖНЫЙ НЮАНС: Встроенные механизмы защиты под Windows (WIN-01..WIN-22)

1) Если антивирус Windows Defender кратковременно блокирует файл при сохранении карты проекта, атомарная запись автоматически выполняет до 5 повторов с задержкой. 2) Зависшие дочерние процессы в песочнице убиваются всем деревом через нативные `Win32 Job Objects`. 3) Поскольку файловая система NTFS нечувствительна к регистру, движок патчей заранее блокирует попытку создать `src/Core.py`, если уже существует `src/core.py` (`CaseCollisionError`). 4) Старайтесь размещать папку проекта ближе к корню диска (например, `C:\dev\orchestra`), чтобы пути не превышали порог 240 символов.

2.3. Запуск в Docker Compose

Docker Compose (с пробросом папки ./runtime на хост)

```
# Сначала создайте .env через python setup_env.py, затем поднимите контейнер:
docker compose up --build -d

# Просмотр логов и остановка:
docker compose logs -f
docker compose down
```

03. Интерактивная настройка через setup_env.py (правило пробела)

Чтобы вам не приходилось вручную редактировать конфигурационные файлы и следить за кавычками в JSON-массивах, в корне репозитория создан интерактивный скрипт `setup_env.py` (также доступен командой `python -m orchestra setup`). Он сам находит созданный .venv, по очереди запрашивает нужные параметры, маскирует ключи на экране и проверяет итоговый файл .env через строгую схему Pydantic.

Запуск мастера первичной настройки

```
# На Linux / macOS:
python3 setup_env.py

# На Windows (или из активированного .venv):
python setup_env.py
```

3.1. Главное правило ввода: как работает ПРОБЕЛ

На любом шаге мастера отправка одного пробела (« » + Enter) или просто пустого Enter означает: «оставить надёжное значение по умолчанию / пропустить необязательное поле». При вводе списка API-ключей программа запрашивает Ключ #1, Ключ #2, Ключ #3 и так далее до 20 слотов: как только вы ввели столько ключей, сколько хотите сейчас (хоть 1, хоть 3), на следующем запросе просто нажмите **Пробел + Enter** — цикл ввода ключей мгновенно завершится, и все остальные пустые слоты прокликивать не нужно!

Шаг мастера	Что вводить и какие есть нюансы
`[1/6] Base URL`	Адрес OpenAI-совместимого сервера. Пробел оставляет NVIDIA Build (https://integrate.api.nvidia.com/v1). Для OpenRouter укажите https://openrouter.ai/api/v1, для DeepSeek — https://api.deepseek.com/v1.
`[2/6] Одна модель`	Точный строковый ID одной модели, доступной на вашем сервере (например, deepseek-ai/deepseek-v3.2 или nvidia/nemotron-3.5-lightning-30b-a3b). Именно эта модель будет вести Архитектора, разработчиков и ревью.

Шаг мастера	Что вводить и какие есть нюансы
`[3/6] API-ключи`	Поочерёдный ввод ключей (#1, #2, #3...). Дубликаты автоматически отбрасываются. На экран выводится только маска вида nvapi-...9x2A. Когда ввели все имеющиеся ключи — отправьте пробел. Если отправить пробел уже на Ключе #1, включится безопасный офлайн-режим provider=mock.
`[4/6] Папка проектов`	Локальный путь хранения всех создаваемых проектов. Пробел оставляет ./runtime. Можно указать абсолютный путь на другом диске (например, D:/OrchestraData).
`[5/6] Telegram-бот`	Необязательно. Отправьте пробел, если будете работать через терминал или браузер. Если введёте токен бота — мастер по очереди запросит ваши числовые Telegram User ID (завершение списка — тоже пробел).
`[6/6] Web-пароль`	Необязательно. Если отправить пробел — дашборд привяжется только к вашему ПК (127.0.0.1:8000) без пароля. Если задать пароль — откроется доступ в локальной сети (0.0.0.0:8000) по Bearer-токену.

● ПРАКТИЧЕСКИЙ СОВЕТ: Повторный запуск мастера никогда не уничтожит старый .env бесследно

Если файл `.env` уже существует, перед записью новых настроек мастер автоматически скопирует прежнюю версию в `.env.bak` (он тоже добавлен в `.gitignore`) и выставит закрытые права доступа `0600` на Linux/macOS.

04. Пул API-ключей, лимиты HTTP 429 и выбор LLM-модели

ORCHESTRA выполняет параллельную сборку независимых веток графа задач, поэтому в систему встроен промышленный менеджер ключей `KeyPoolManager`. Понимание его нюансов позволит вам комфортно работать как с одним бесплатным ключом, так и с пулом из 5–20 ключей.

4.1. Нюанс работы с ОДНИМ или ДВУМЯ ключами

По умолчанию оркестратор разрешает до 6 параллельных агентов (`MAX_CONCURRENT_AGENTS=6`) и до 2 одновременных запросов на один ключ (`MAX_CONCURRENT_PER_KEY=2`). Если у вас пока только **1 API-ключ** с жёстким лимитом запросов в минуту (например, бесплатный тариф NVIDIA Build ~40 RPM), пул сам выстроит агентов в аккуратную очередь по 2 параллельных потока. Если же вы замечаете частые паузы из-за лимита RPM провайдера, добавьте в конец файла .env строку:

Рекомендуемая настройка в .env при работе всего с 1 ключом на бесплатном тарифе

```
ORCHESTRA_KEY_POOL__MAX_CONCURRENT_PER_KEY=1
ORCHESTRA_SWARM__MAX_CONCURRENT_AGENTS=2
```

4.2. Как пул автоматически переживает ошибки 429 и сбои сети

- **ACTIVE** — ключ здоров и принимает задачи агентов.
- **COOLING** — сервер вернул `HTTP 429 Too Many Requests` или `5xx`. Ключ уходит на «остывание» ровно на время заголовка Retry-After (или 60 сек для 429 / 15 сек для 5xx), а запрос агента без падения прозрачно перенаправляется на другой здоровый ключ пула (или ждёт окончания паузы, если ключ один).
- **DISABLED** — сервер вернул `401/403 AuthFailed` (ключ отозван или неверный) либо произошло 5 сбоев подряд. Такой ключ исключается из ротации до перезапуска.

4.3. Нюансы выбора модели (DEFAULT_MODEL)

> ● **ВНИМАНИЕ: Точное совпадение имени модели на сервере провайдера**
> При старте боевой сборки оркестратор выполняет быстрый проверочный запрос (`CapabilityProber`: `GET /models` + микро-стрим на 8 токенов), чтобы убедиться, что выбранная модель реально существует у вашего провайдера, узнать её поле рассуждений (reasoning_content) и минимально допустимую температуру. Если вы скопировали имя модели с опечаткой — система сразу остановится с понятной ошибкой `ModelNotFoundError`, сэкономив ваше время и токены.

Лучше всего для роя подходят модели с сильным структурным JSON-выводом и режимом рассуждений (thinking): линейки DeepSeek V3.2 / R1, Qwen 3 (235B / 32B), Nemotron Ultra / Lightning. Двухпотоковый маршрутизатор `DualStreamRouter` физически отделяет поток мыслей от итогового кода — рассуждения модели никогда не попадут внутрь генерируемых .py файлов!

05. Самодиагностика (doctor) и офлайн-проверка (demo)

Перед первым боевым запуском обязательно выполните две быстрые проверочные команды. Они занимают меньше двух секунд и подтверждают, что Python, Git, песочница, кодировка UTF-8, переводы строк LF и все 32 встроенных скилла полностью исправны.

Предполётная проверка

```
# 1. Полная диагностика платформы и конфигурации с вердиктом на русском:
.venv/bin/python -m orchestra doctor

# 2. Контрольная офлайн-сборка 34-узлового крипто-проекта без траты ключей:
.venv/bin/python -m orchestra demo
```

На что обратить внимание в таблице команды doctor: строка «Платформа (Git)» должна быть в статусе OK, строка «Пул API-ключей» после запуска setup_env.py должна показывать количество введённых вами ключей (> 0 для боевого режима), а «Реестр /skills/» — 20 системных скиллов (плюс 12 крипто-скиллов подгружаются автоматически при сборке крипто-проектов).

06. Как правильно писать ТЗ и запускать сборку (orchestra run)

Команда запуска реальной разработки выглядит максимально просто и принимает либо текст технического задания в кавычках, либо путь к текстовому/Markdown-файлу с подробным ТЗ:

Боевой запуск фабрики

```
# Вариант А: Короткое ТЗ прямо в командной строке
.venv/bin/python -m orchestra run "Напиши библиотеку расчёта комиссий и PnL на Decimal с pytest-те
стами" --project-id pnl_lib

# Вариант Б: Подробное ТЗ из подготовленного файла (РЕКОМЕНДУЕТСЯ для крупных задач!)
.venv/bin/python -m orchestra run ./my_project_spec.md --project-id order_router_v1

# Проверить статус всех узлов графа в любой момент:
.venv/bin/python -m orchestra status --project-id order_router_v1
```

6.1. Как составить идеальное ТЗ: 6 практических правил

- **1. Чётко называйте входы, выходы и типы данных**: например, «функция принимает список сделок в JSON, считает средневзвешенную цену входа через Decimal и возвращает Pydantic-модель отчёта».

- **2. Сразу фиксируйте бизнес-константы и лимиты**: агентам строжайше запрещено выдумывать тарифы, комиссии, тайм-ауты и лимиты риска «из головы». Если они важны — напишите их прямо в ТЗ, иначе Архитектор поставит узел на паузу и пришлёт вам вопрос.
- **3. Соблюдайте модульность (атомы до 300 строк)**: в ORCHESTRA действует жёсткий закон `MAX_CHUNK_LINES = 300`. Если Судья увидит файл длиннее 300 строк, он отклонит патч и заставит агента разбить код на чистые небольшие модули.
- **4. Указывайте желаемый стек и ограничения**: например, «использовать только стандартную библиотеку Python 3.11 + Pydantic v2, хранение в SQLite, без внешних API».
- **5. Давайте осмысленный --project-id латиницей без пробелов**: например, `arb_scanner_01`. Если запустить команду повторно с тем же `--project-id`, система подхватит существующую папку в `runtime/projects/<id>/`. Для нового проекта с нуля давайте новый ID!
- **6. Описывайте краевые сценарии**: пустой ввод, отрицательные значения, дубликаты сообщений, обрыв связи — QA-агент превратит их в обязательные автотесты.

6.2. Что именно проверяет Независимый Судья на каждом узле

Агент-разработчик структурно лишён права пометить свой узел как DONE (попытка сделать это вызывает исключение `IndependentReviewViolationError`). Он может лишь перевести узел в VERIFYING, после чего включается автоматический конвейер Судьи:

- `PatchEngine` проверяет, что агент изменил **только файлы из своего списка owned_files** (чужие файлы трогать запрещено!), нет коллизий регистра имён и размер каждого файла <= 300 строк.
- `ast.parse` + `ruff check` проверяют синтаксическую чистоту и стандарты стиля.
- `pytest -q` запускает тесты в изолированной песочнице без передачи ваших секретов из .env.
- Для крипто-проектов дополнительно прогоняются все 13 AST-правил безопасности денег.
- Если любая проверка упала — дефекты записываются в узел, и агент получает до 2 попыток точечного исправления. Если всё зелёное — делается атомарный git commit!

07. Полный справочник всех команд CLI

Команда (вызов через .venv/bin/python -m ...)	Назначение и результат
`python setup_env.py` `orchestra setup`	Пошаговый интерактивный мастер создания .env (модель, ключи по очереди, пробел для пропуска).
`orchestra doctor`	Полная диагностика ОС, UTF-8/LF, ResourceLimiter, Git, песочницы, провайдера, ключей и скиллов.
`orchestra init`	Предварительная инициализация структуры ./runtime/ и манифеста встроенных скиллов.
`orchestra demo`	Эталонный офлайн-прогон 34-узлового крипто-проекта (PAPER) на MockProvider без ключей.
`orchestra run "<ТЗ>"` `--project-id ID [--mock]`	Запуск полного цикла сборки проекта по тексту ТЗ или пути к файлу спецификации.
`orchestra status` `--project-id ID`	Таблица всех узлов MentalMap проекта: типы, текущие статусы, зависимости и расход токенов.
`orchestra keys`	Публичная статистика пула ключей (ID-отпечаток, статус ACTIVE/COOLING, запросы, счётчик 429/5xx).
`orchestra skills [--crypto]`	Список всех активных системных скиллов (с флагом --crypto добавляет и 12 крипто-скиллов).
`orchestra talents`	Таблица накопленных профилей профессий: версия профиля, процент сдачи с 1-й попытки, число проектов.
`orchestra serve` `--project-id ID --port 8000`	Запуск локального HTTP + WebSocket сервера и офлайн Web-дашборда оператора.
`orchestra version`	Вывод текущей версии установленного пакета ORCHESTRA.

08. Веб-консоль оператора (serve) и ответы на эскалации

Веб-консоль оператора позволяет визуально наблюдать за графом узлов, читать живой поток рассуждений агентов, проверять состояние ключей и отвечать на вопросы Архитектора прямо в браузере. Все скрипты и стили лежат локально в `orchestra/web/` — интерфейс не тянет ни одного внешнего CDN-скрипта и мгновенно открывается даже без интернета.

Запуск Web-дашборда

```
# Безопасный локальный запуск только для вашего компьютера:
.venv/bin/python -m orchestra serve --project-id order_router_v1 --host 127.0.0.1 --port 8000

# Затем откройте в браузере: http://127.0.0.1:8000
```

8.1. Как работают неблокирующие эскалации (BLOCKED_HUMAN)

В ORCHESTRA трёхуровневая система принятия решений (L1 -> L2 -> L3): если ответ есть в карте или дневнике (L1) — агент берёт его молча; если вопрос чисто технический (L2, например выбор структуры индекса) — Архитектор решает его сам в один проход и пишет ADR; и только если вопрос касается ваших денег, бизнес-логики или исчерпаны все попытки исправления узла (L3) — узел переходит в статус `BLOCKED_HUMAN`.

> ● **ВАЖНЫЙ НЮАНС: Нулевой простой соседних веток (Non-Blocking Escalation)**
> Пока один спорный узел ждёт вашего ответа в статусе BLOCKED_HUMAN, **все остальные независимые ветви проекта продолжают собираться и тестироваться**! Ответить на открытый вопрос можно в панели эскалаций Web-консоли, в Telegram или POST-запросом на `/api/v1/questions/{question_id}/answer`.

> ● **БЕЗОПАСНОСТЬ И ДЕНЬГИ: Доступ к Web-консоли по сети (0.0.0.0)**
> Если вы запускаете сервер на внешнем интерфейсе (--host 0.0.0.0), обязательно задайте надёжный пароль в `ORCHESTRA_WEB__ACCESS_TOKEN`. Тогда все эндпоинты `/api/v1/*` и WebSocket `/ws?token=...` будут требовать авторизацию, при этом сырые API-ключи никогда не возвращаются сервером даже авторизованному оператору!

09. Telegram-бридж оператора: настройка и нюансы безопасности

Telegram-мост удобен, когда вы запускаете крупную сборку и хотите получать краткие русскоязычные сводки (SITREP) с кнопками выбора вариантов прямо на телефон.

- **Шаг 1**: создайте нового бота через официального `@BotFather` и скопируйте выданный токен вида 123456789:AAH...
- **Шаг 2**: узнайте свой числовой Telegram User ID (это только цифры, например 482910422, а НЕ @username!).
- **Шаг 3**: запустите `python setup_env.py` и укажите токен и свой User ID на шаге [5/6].

> ● **БЕЗОПАСНОСТЬ И ДЕНЬГИ: Жёсткий режим Fail-Closed в Telegram**
> Если список `ORCHESTRA_TELEGRAM__ALLOWED_USER_IDS` пуст (`[]`), мост автоматически отвергает сообщения абсолютно от всех пользователей интернета, включая вас самих! Это сделано специально, чтобы никто посторонний не мог управлять вашей фабрикой, угадав имя бота. Таймаут ожидания вашего ответа по умолчанию равен 3600 сек (1 час), после чего применяется безопасное действие `pause_node` (пауза узла без самодеятельности).

10. Накопление опыта: Скиллы (/skills/) и Таланты (/agent_talents/)

Главное отличие ORCHESTRA от одноразовых генераторов кода — двухконтурная долговременная память, которая сохраняется на вашем диске между проектами:

10.1. Реестр исполняемых скиллов (/skills/) и Zero Token Waste

В поставку входят 20 системных скиллов (чтение AST без мусора, точечное наложение патчей, проверка реального существования API-символов против галлюцинаций, поиск по документации BM25, анализ мёртвого кода, сканер утечек секретов, атомарный git-коммит и др.) плюс 12 доменных крипто-скиллов. Агенты видят в своём промпте только крошечный индекс имён и сигнатур (до 600 токенов) — исходный код скиллов никогда не забивает контекстное окно модели! Сам скилл запускается оркестратором локально в отдельном подпроцессе с очищенным окружением.

Любое обновление скилла проходит через `SkillMutationGate` и статический сканер безопасности `security_scan.py` (запрещены eval, exec, shell=True, pickle, прямые сокеты). Если регрессионный тест падает — изменение отвергается или скилл изолируется.

10.2. Профили профессий и Empirical Proof Gate дневников

После завершения каждой сборки модуль `PostMortemAnalyzer` анализирует реальные ретраи и ошибки проекта, обновляя профили ролей в runtime/agent_talents//. В дневник открытий (`DISCOVERY_DIARY.md`) со статусом `VERIFIED` попадают только те уроки, которые имеют эмпирическое доказательство из песочницы (команда, exit_code == 0, SHA-256 артефакта). Сырые догадки (HYPOTHESIS) хранятся для истории, но никогда не подмешиваются в системные промпты будущих проектов.

11. Крипто-домен: мультибиржевая база, Decimal и защита LIVE-режима

В ORCHESTRA встроен специализированный доменный пакет `CryptoTradingDomainPack` для построения мультибиржевых торговых платформ и исследовательских бэктестеров. Он включается автоматически на стадии S1, как только в вашем ТЗ встречаются минимум 2 профильных термина (например: crypto, binance, bybit, okx, orderbook, ohlcv, funding rate, backtest, slippage, крипто, биржа, фьючерсы, свечи, ордер).

11.1. 13 жёстких AST-правил, которые Судья проверяет механически

Код правила AST	От какой финансовой ошибки защищает
`NO_FLOAT_MONEY`	Полный запрет типа float для цен, объёмов, балансов, комиссий и PnL — только Decimal из строк.
`NO_NAIVE_DATETIME`	Запрет datetime.now() без часового пояса и utcnow() — всё время строго в UTC.
`NO_WALLCLOCK_IN_STRATEGY`	Стратегии запрещено читать системные часы ПК (только время биржевого события — для единства с бэктестом).
`NO_DIRECT_EXCHANGE_IMPORT`	Стратегия не имеет права импортировать ccxt напрямую — работа только через адаптер шлюза.
`NO_LOOKAHEAD`	Запрет заглядывания в будущее (например, .shift(-1)) в расчёте сигналов и бэктесте.
`MANDATORY_CLIENT_ORDER_ID`	Каждая отправка ордера обязана содержать уникальный идемпотентный client_order_id.
`ORDER_AUDIT_REQUIRED`	Обязательная запись намерения в append-only журнал аудита до сетевой отправки.
`NO_UNBOUNDED_AVERAGING`	Запрет бесконтрольного мартингейла и усреднения без явного риск-лимита.
`DECIMAL_JSON_SAFETY`	Запрет сериализации Decimal через float в JSON (только строковое представление).
`SECRET_NOT_LITERAL`	Запрет хардкода api_key / api_secret строковыми литералами в исходном коде.
`LIVE_MODE_GUARD`	Блокировка обращения к TradingMode.LIVE без явного флага подтверждения человека.
`NO_EXCHANGE_LOGIC_OUTSIDE_ADAPTER`	Запрет утечки специфичных заголовков конкретной биржи за пределы её адаптера.
`NO_FLOAT_CROSS_EXCHANGE`	Запрет float-арифметики при кросс-биржевом сведении спредов и позиций.

● БЕЗОПАСНОСТЬ И ДЕНЬГИ: Правила работы с биржевыми ключами и режимом LIVE (прочитайте обязательно!)

1) Режим по умолчанию во всех генерируемых торговых проектах — строго **PAPER** (бумажная симуляция без отправки реальных ордеров), а список активных бирж в .env пуст (`ORCHESTRA_EXCHANGES__ENABLED=[]`).
2) Скилл подписи запросов `hmac_sign_request` принимает **только имя переменной окружения** (`secret_ref`), но не сам секрет — биржевой ключ никогда не проходит через LLM.
3) Когда вы в будущем перейдёте к проверке созданного робота на реальной бирже: сначала прогоните его минимум на TESTNET / SHADOW, создавайте биржевые ключи **СТРОГО БЕЗ ПРАВА ВЫВОДА СРЕДСТВ (Withdraw = OFF)** и с привязкой к вашему IP. Программное обеспечение предоставляется в инженерных и образовательных целях и не является инвестиционной рекомендацией.

11.2. Настройка списка бирж в exchanges.yaml

В корне проекта лежит мультибиржевой реестр `exchanges.yaml` с профилями возможностей (точность тиков, веса лимитеров, интервалы сверки и пороги рассинхрона часов). У каждой биржи свой изолированный лимитер запросов и независимый аварийный рубильник (`KillSwitch`): перегрузка одной биржи не роняет соседние.

12. Бюджеты токенов, тайм-ауты и автовосстановление после сбоев

- **Бюджет контекста 20 / 30 / 50**: каждый промпт собирается с коэффициентом запаса 0.85 от окна модели: 20% отводится на устав роли и проверенные уроки дневника, 30% — на локальную суб-карту узла и замороженные контракты соседей, 50% — на индекс скиллов, рабочие диффы и скользящее окно последних 6 шагов (более старые шаги сжимаются в компактные дайджесты, поэтому агент не переполняет память на длинных задачах).
- **Ограничители расходов**: на один узел по умолчанию выделяется до `200 000` токенов (`ORCHESTRA_BUDGETS__NODE_TOKEN_LIMIT`). Вы также можете задать жёсткий потолок токенов на весь проект целиком через `ORCHESTRA_BUDGETS__PROJECT_TOKEN_LIMIT` и лимит времени в секундах через `ORCHESTRA_BUDGETS__PROJECT_WALLCLOCK_S`.
- **Корректная остановка (Ctrl+C) и Crash Recovery**: при нажатии Ctrl+C `GracefulShutdownCoordinator` даёт активным задачам до 30 секунд аккуратно сохранить контрольную точку. А если компьютер внезапно выключился по питанию, при следующем старте `StateStore` проверит SHA-256 чексумму `mental_map.json`, при повреждении возьмёт последний целый .bak-снапшот, доиграет хвост событий из `events.jsonl`, а `CheckpointManager` вернёт зависшие узлы обратно в очередь READY — все уже готовые узлы DONE сохранятся!

13. Справочник переменных .env и решение типовых проблем (FAQ)

13.1. Ключевые параметры файла .env

Переменная .env	По умолчанию	Описание и рекомендации
ORCHESTRA_PROFILE	dev	Профиль окружения: dev | ci | prod (в prod маскирование секретов принудительно неотключаемо).
ORCHESTRA_RUNTIME_DIR	./runtime	Локальная папка для всех проектов, логов, кэша, талантов и скиллов.
ORCHESTRA_LLM__PROVIDER	openai_compat	Боевой режим openai_compat или офлайн-режим mock без ключей.
ORCHESTRA_LLM__BASE_URL	https://integrate.api.nvidia.com/v1	Базовый URL вашего OpenAI-совместимого провайдера.
ORCHESTRA_LLM__DEFAULT_MODEL	nvidia/nemotron-3.5-...	Точный ID одной рабочей модели LLM.
ORCHESTRA_KEY_POOL__KEYS	["ключ1","ключ2"]	JSON-массив ваших API-ключей (автоматически заполняется setup_env.py).
ORCHESTRA_KEY_POOL__MAX_CONCURRENT_PER_KEY	2	Максимум параллельных запросов через один ключ (поставьте 1 при жёстком RPM).
ORCHESTRA_SWARM__MAX_CONCURRENT_AGENTS	6	Максимум одновременно работающих агентов роя.
ORCHESTRA_SANDBOX__MODE	subprocess	Режим песочницы: subprocess (быстро) или docker (полная контейнеризация).
ORCHESTRA_TELEGRAM__ENABLED	false	Включение Telegram-бриджа (требует BOT_TOKEN и непустой ALLOWED_USER_IDS).
ORCHESTRA_WEB__HOST / PORT	127.0.0.1 / 8000	Адрес и порт локальной веб-консоли оператора.

13.2. Быстрое решение частых проблем (Troubleshooting)

Симптом / сообщение	Точная причина	Что сделать оператору
ConfigurationError: пул API-ключей пуст	Вы запустили боевой orchestra run без флага --mock, но не добавили ни одного ключа.	Запустите python setup_env.py и введите хотя бы 1 API-ключ (или добавьте флаг --mock для теста).
ModelNotFoundError при старте	Указанная в DEFAULT_MODEL модель отсутствует на выбранном вами BASE_URL.	Проверьте точное имя модели в личном кабинете провайдера и перезапустите python setup_env.py.
Частые переходы ключа в COOLING (429)	Превышен минутный лимит запросов (RPM) на одном бесплатном ключе.	Уменьшите в .env: `MAX_CONCURRENT_PER_KEY=1` и `MAX_CONCURRENT_AGENTS=2` либо добавьте 2-й ключ.
Узел остановился в BLOCKED_HUMAN	В ТЗ не хватает важного бизнес-правила либо узел исчерпал 2 попытки авто-починки.	Откройте Web-дашборд (orchestra serve) или статус узла, посмотрите вопрос/дефекты и дайте ответ.
Судья отклоняет код: File has > 300 lines	Сработало правило атомарности кода MAX_CHUNK_LINES = 300.	Штатный процесс: на следующем ретрае агент сам разобьёт крупный модуль на компактные файлы.
Telegram-бот молчит и не отвечает	Ваш числовой ID не внесён в ALLOWED_USER_IDS (сработала защита fail-closed).	Укажите свой цифровой Telegram User ID в шаге [5/6] setup_env.py и перезапустите процесс.
Как проверить, что я случайно не сохранил секрет в коде?	Перед публикацией любых правок нужно убедиться в отсутствии ключей в файлах и коммитах.	Запустите встроенный сканер: .venv/bin/python scripts/check_secrets_leak.py . (должно быть 0 findings).

● ПРАКТИЧЕСКИЙ СОВЕТ: Шпаргалка полного рабочего цикла в 4 команды

1) `make install` — один раз развернуть окружение.
2) `python setup_env.py` — ввести модель и нужное число ключей (остальное — пробел).
3) `.venv/bin/python -m orchestra doctor` — убедиться, что все системы зелёные.
4) `.venv/bin/python -m orchestra run spec.md --project-id my_app` — собрать готовый проверенный проект в `./runtime/projects/my_app/workspace/`!